

DATA-STRUCTURES & ALGORITHMS

with

Manoj Prabhakaran

Lecture 22

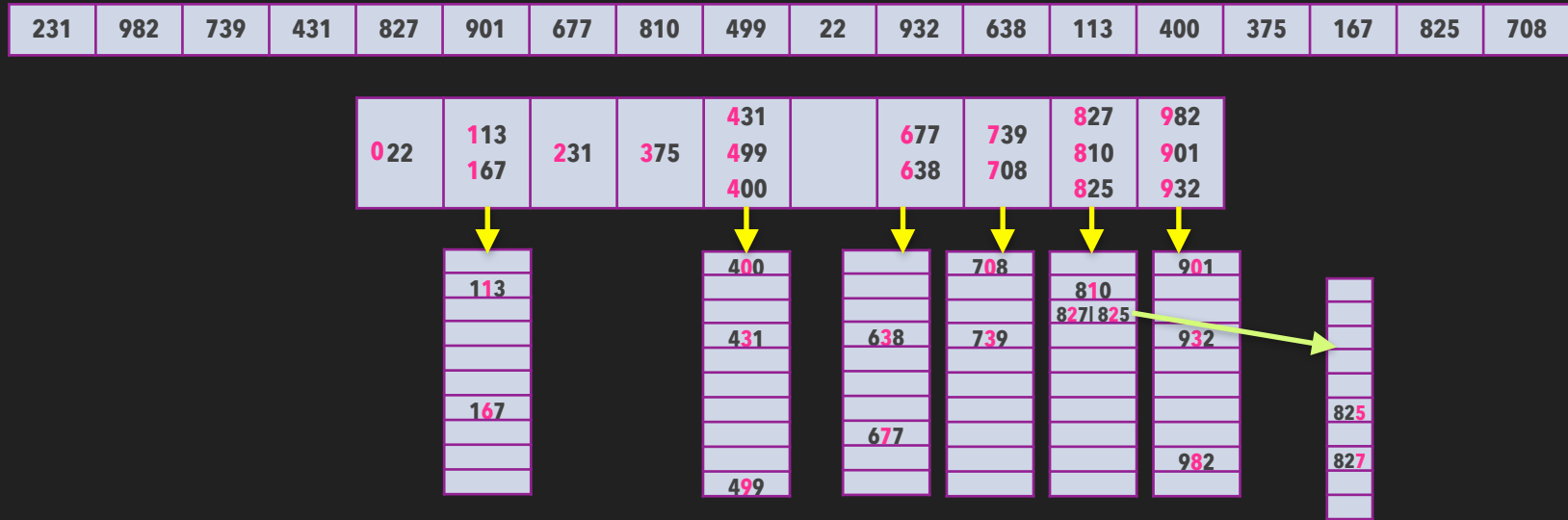
Radix Sort: MSD vs. LSD

Sorting

- Last time:
 - Counting sort: When the key-space is small, calculate the offsets for items with each key, and move them to the correct position
 - Bucket sort: Counting sort, applied to bucketed keys, followed by sorting each bucket
 - MSD Radix Sort: Bucket sort, with large buckets sorted recursively
- Today:
 - MSD Radix Sort (continued)
 - Least-Significant Digit (LSD) Radix Sort
 - MSD vs LSD Radix sort

Most-Significant-Digit Radix Sort

- Consider D-digit numbers
 - First collect by MSD



- Each bucket with more than one item is sorted recursively using the remaining digits

Most-Significant-Digit Radix Sort

- What MSD Radix sort implements is lexicographic sorting (dictionary order)
 - E.g., $\text{cab} < \text{car} < \text{dad} < \text{dog}$
 - Strings needn't be the same length: $\text{dog} < \text{dogma}$ (imagine an end-of-string character lesser than all other characters)
- For integers, to ensure lexicographic ordering is the same as numerical ordering, need 0-padding so that all numbers have the same number of digits
 - E.g., $22 < 113$ numerically but not lexicographically; after 0-padding, lexicographically also, $022 < 113$
 - "Digit" can be in any convenient base, e.g., base 256 allows each byte to be a single digit

Most-Significant-Digit Radix Sort

- There are B^D D-digit numbers (or D-long strings) in base B
 - This space is exponentially large in D, making counting sort impractical even for modest D
 - Also, comparison sort is expensive for large D, since a single comparison of two keys requires inspecting up to D digits, one by one
- MSD Radix sort costs:
 - Worst-case time and space: $O(D(n+B))$
 - Exercise: Prove by induction on D. Base case, $D=1$, is counting sort.

LSD Radix Sort

- What if we first sort the numbers by their LSD?

	1000	100	10	1		1000	100	10	1		1000	100	10	1		1000	100	10	1		1000	100	10	1		
4012	4	0	1	2		0	0	3	0		7	7	0	3		4	0	1	2		0	0	3	0		30
1091	1	0	9	1		1	0	9	1		6	9	0	5		0	0	3	0		0	0	9	2		92
30	0	0	3	0		4	0	1	2		4	0	1	2		1	0	9	1		0	7	8	5		785
785	0	7	8	5		9	4	5	2		0	0	3	0		0	0	9	2		1	0	9	1		1091
7703	7	7	0	3		0	0	9	2		9	4	5	2		1	1	7	9		1	1	7	9		1179
4388	4	3	8	8		7	7	0	3		1	1	7	9		4	3	8	8		3	4	9	8		3498
9452	9	4	5	2		0	7	8	5		0	7	8	5		9	4	5	2		4	0	1	2		4012
92	0	0	9	2		6	9	0	5		4	3	8	8		3	4	9	8		4	3	8	8		4388
1179	1	1	7	9		4	3	8	8		1	0	9	1		7	7	0	3		6	9	0	5		6905
6905	6	9	0	5		3	4	9	8		0	0	9	2		0	7	8	5		7	7	0	3		7703
3498	3	4	9	8		1	1	7	9		3	4	9	8		6	9	0	5		9	4	5	2		9452

LSD Radix Sort

- LSD Radix sort: Sorting all the numbers by their digits using counting sort, one digit at a time, starting from LSD
- Why does LSD work? Proof by induction:
 - Base case is the 1 digit case.
 - Enough to argue that any two numbers are correctly sorted relative to each other
 - If they differ in MSD, the last pass sorts them correctly
 - If they have the same MSD, then they were correctly sorted before the last pass (by induction) and the last pass does not change their relative order (we crucially need each pass to be a stable sort)

LSD Radix Sort

- Simply call counting sort in a loop, one digit at a time as the key
 - Here each item is a D-sized vector of B-ary digits

```
void LSD_Radix(std::vector<std::vector<int>>& A, int D, int B) {
    std::vector<std::vector<int>> aux(A.size());
    for(int d=0; d<D; ++d) { // counting sort using d-th digit (d=0 is LSD)
        std::vector<int> offset(B,0);
        for(auto& x : A) ++offset[x[d]]; // after this, offset has counts
        for(unsigned int i=0, sum = 0; i < B; i++) {
            unsigned int tmp = offset[i];
            offset[i] = sum; sum += tmp;    // offset[i] has sum of previous counts
        }
        for(auto& x : A) // aux[ offset[i] ] is where next item with key i goes
            aux [ offset[x[d]]++ ] = std::move(x); // move items from A to aux
        A.swap(aux); // finally move aux vector to A, keeping aux's size same
    }
}
```


MSD vs. LSD Radix Sort

- MSD version can eliminate short strings (or strings that fall into small buckets) early on. LSD version goes over every string in all D iterations.
 - MSD radix sort better suited for sorting variable length strings, or long strings that are lexicographically "far apart"
- MSD version requires more book-keeping, as sorting is applied to each bucket separately. LSD version is simpler and more cache-friendly.
 - LSD radix sort better suited for sorting fixed-length "strings" (e.g., 32-bit integers with exactly $D = 4$ digits, in base $B = 2^8 = 256$)
- Both are highly parallelizable, but the LSD version allows all processors working equally and in-sync
 - LSD radix sort (with optimisations) used for sorting integers in GPUs

Summary

- Counting sort
 - Useful when the key domain size k is small. $O(n+k)$ time and auxiliary space
- Bucket sort
 - If data is “well-spread” over a few buckets into which keys can be partitioned
 - On well-spread data, using k buckets, $O(n+k)$ time and auxiliary space. But worst-case time is $O(n \log n + k)$ (assuming $O(1)$ -time comparisons).
- MSD Radix Sort
 - Using bucket sort recursively, for lexicographically sorting strings. Worst-case time and auxiliary space: $O(D(n+B))$ for D -long strings over an alphabet of size B . (Faster, if common prefix of every pair is short.)
- LSD Radix Sort
 - Using counting sort iteratively. $O(D(n+B))$ time and auxiliary space.
 - Parallel optimised version used for sorting numbers in modern GPUs